

B⁺-Tree Deletion: Explanation and Examples

1 Introduction

Deleting a key from a B⁺-tree is more involved than insertion because it can create a **node underflow**, where a node contains fewer keys than the minimum required. To maintain the B⁺-tree properties, underfull nodes must be corrected using one of the following strategies:

- **Redistribution (Borrowing):** Transfer keys from a sibling node to restore the minimum occupancy without merging.
- **Merging:** Combine the underfull node with a sibling and remove the corresponding separator key from the parent.

Deletion can propagate upward:

- If a merge causes the parent to become underfull, it may need redistribution or merging as well.
- In the worst case, the root may lose keys, potentially decreasing the tree height by one level.

Despite its complexity, the deletion operation preserves the **balance** of the tree and guarantees logarithmic performance for searches, insertions, and deletions.

Step 1: Initial Tree Before Deletion

The tree before deleting “Srinivasan”:

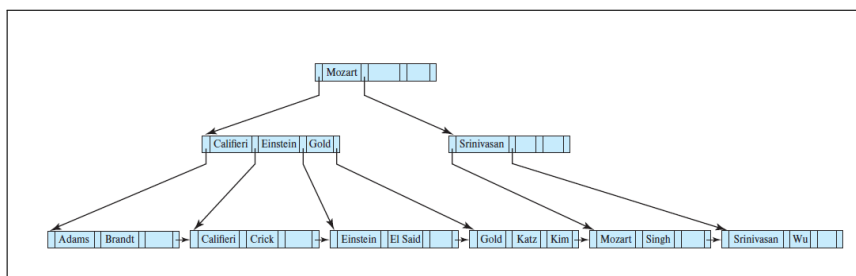


Figure 1: Initial B⁺-tree before deletion of “Srinivasan”

Step 2: Deleting “Srinivasan”

Step-by-Step Deletion of “Srinivasan”

1. Locate “Srinivasan” using the lookup algorithm.
2. Remove “Srinivasan” from its leaf node.
3. Check leaf occupancy: if it has fewer than $\lceil (n-1)/2 \rceil$ keys, an underflow occurs.
4. Fix underflow by redistributing keys with a sibling and updating the parent separator key.

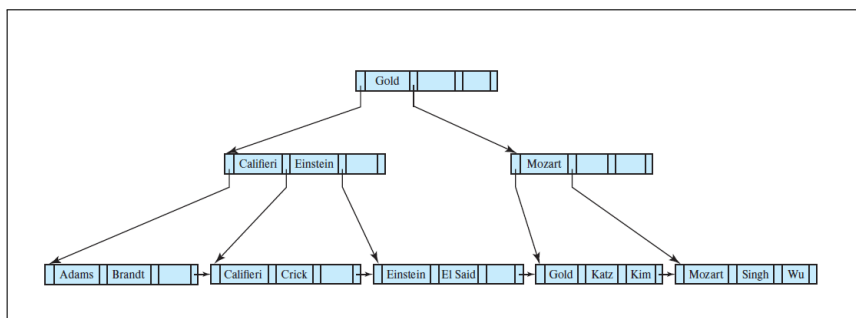


Figure 2: Deletion of “Srinivasan” with redistribution to sibling

Step 3: Deleting “Singh” and “Wu”

Deletion Details

- Deleting “Singh” does not cause underflow; no restructuring is required.
- Deleting “Wu” causes the leaf to become underfull:
 - Redistribute keys from sibling (e.g., move “Kim” between nodes).
 - Update parent separator key accordingly.

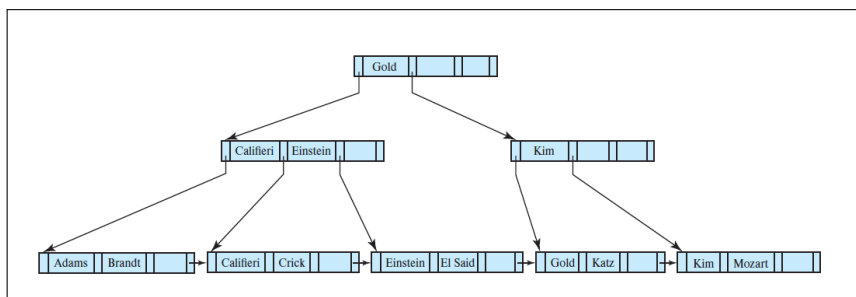


Figure 3: Deletion of “Singh” and “Wu” with redistribution

Step 4: Deleting “Gold”

B⁺ Tree Deletion Example

1. **Delete the key:** Remove “Gold” from its leaf node.
2. **Check underflow:** If the leaf still satisfies the minimum occupancy, the process ends. Otherwise, the leaf is *underfull* and requires adjustment.
3. **Borrow from a sibling (preferred):**
 - Identify a sibling (left or right) with more than the minimum number of keys.
 - **From the left sibling:** take the largest key and insert it into the underfull leaf. Update the parent separator key to reflect the new smallest key in the underfull leaf.
 - **From the right sibling:** take the smallest key and insert it into the underfull leaf. Update the parent separator key accordingly.
4. **Merge if borrowing is not possible:**
 - Merge the underfull leaf with a sibling.
 - **If merging with the left sibling:** remove the separator key in the parent that separates the two nodes.
 - **If merging with the right sibling:** the separator key moves down into the merged node or remains in the parent, depending on implementation.
5. **Recursive adjustment:** If removing a parent key causes the parent to become underfull, apply borrowing or merging recursively up to the root.
6. **Root adjustment:** If the root ends up with only a single child, reduce the tree height by making that child the new root.

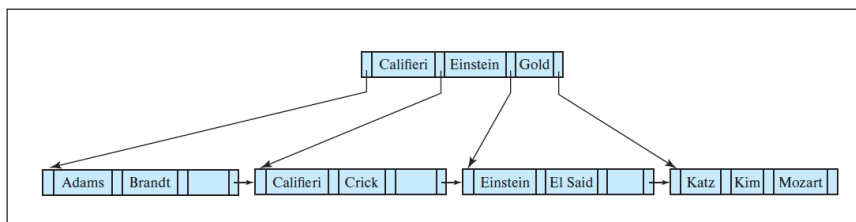


Figure 4: Deletion of “Gold” and merging of nodes

General Deletion Algorithm

Algorithm: Deletion from B⁺-Tree

procedure delete(key K):

1. Locate leaf L containing K .
2. Remove K from L .
3. If L has enough keys, stop.
4. Otherwise:
 - 4.1. Try to redistribute keys with a sibling and update parent separator key.
 - 4.2. If redistribution is not possible, merge with sibling and remove the separator from parent (only if merging with left sibling; otherwise handle appropriately).
5. If parent becomes underfull, recursively apply deletion up to the root.
6. If root has only one child and becomes empty, make the child the new root.

Key Insights

- Deletion may cause **redistribution** or **merging**, but the tree always remains balanced.
- Tree height may decrease if the root has only one child after merging.
- All B⁺-tree properties are preserved after deletion.
- Worst-case time complexity: $O(\log N)$.